NetWolf open source python based network utility

J Arneil & Vishesh Grover - Yoobee College of Creative Innovation - 2026

Abstract:

In this proposal, we outline the development of our proposed open source network analysis and stress testing tool, NETWOLF. This tool is designed to consolidate stress testing and vulnerability scanning into a single utility. Current industry utilities like nmap or hping3 come with steep learning curves and make users utilize multiple disconnected tools while performing network analysis and testing. Netwolf serves to address these inefficiencies by consolidating the workflow into a single tool. Netwolf contains both network scanning and penetration testing utilities within one connected ecosystem, focusing on an open-source and modular design, allowing users and developers to adapt this tool to their requirements. Netwolf also serves as a powerful educational tool by serving as an intuitive and transparent utility, along with providing a controlled environment for learning about penetration testing and network scanning. The proposed development lifecycle for NETWOLF is centred around the creation of a functional Minimum viable product that showcases the ethical applications of the proposed utility.

Introduction:

In this proposal, we outline the development of a custom network tool built using Python. The primary functions of this tool are an integrated network scanner and an in-built DOS tool. The user will then be able to utilise these tools in conjunction with each other to perform vulnerability scanning and stress testing within one program, utilising only the host device's computational resources. This tool falls under the “Threats and Vulnerability Scanning” project scope, utilising the port scanning module, while also testing the security principle of “availability” through the “DOS” module. It therefore provides users with an overview of the security and threat resistance of the applied network. Our motivation for creating this project stems from several identified issues. Firstly, learning about the functionality and creation of network scanning and denial-of-service tools is currently quite restricted, especially considering that AI tools do not assist in the creation of such tools. Therefore, we aim to create an open-source modular toolkit that will serve as both a real-world utility and educational tool. The second issue identified is that the current workflow is disconnected, requiring users to switch between different utilities when performing network analysis and penetration testing. Our tool aims to contain all required functionality inside one modular ecosystem, allowing the user maximum flexibility and efficiency when performing tasks.

Literature Review:

- Abu Bakar and Kijsirikul (2023) detail the development of a Python-based network testing tool. This paper introduces a Data plane development kit-based network analysis tool, which has been created to circumvent the current speed, efficiency, and accuracy limitations of existing network

scanning utilities. This report details how the proposed scanner significantly outperforms existing tools, with a two-fold improvement in scan speed and an inaccuracy rate of only 0.5%. The tool and implementations proposed in this paper present a highly efficient solution for network assessment.

- Ghazali and Hassan (2011) analyze the mechanics of UDP flood Denial of Service (DoS) attacks within IoT networks. This paper examines how susceptible different network implementations and standard protocol specifications are to DOS attacks. The authors have compared these various attacks and concluded that mitigation strategies must be created on an instance-based basis and that there is no true one-size-fits-all method when it comes to preventing these types of attacks.
- Pittman (2023) presents a comparative analysis of popular existing network tools, referencing Nmap, Masscan, and ZMap. The findings of this report state that there is no significant difference in the overall accuracy and performance of the tools mentioned. But there is an apparent difference in their efficiencies. The study highlights that the insights gathered can allow security professionals to select the best available tool based on operational requirements.
- Li et al. (2026) detail the architecture development and the effect of the network system tool XMap. This paper highlights how XMap was the first tool capable of internet-wide IPv6 scanning, along with detailing the impact the tool has made in recent times, including in research instances with XMap being referenced in at least 50 research papers since its release.
- Jafarian et al. (2023) outline the implementation of a proactive inline scan detection system, also known as SDS. This system uses inline scan detection to monitor traffic ingress and egress and detect malicious traffic. This paper concludes that this system can identify all types of scanning behavior, including worms and scanners with conservative timing profiles.

1.2 Existing technologies and methodologies:

Network scanning tools:

Nmap: Nmap is the current industry standard for network analysis and port scanning. It functions by using IP packets to assess the hosts within a network. Returning data such as the operating system and services offered by the host devices.

Masscan: This tool is designed to function at high volumes with extreme speed. Although it is faster than Nmap, it comes with a sacrifice in terms of accuracy. Meaning that it is usually used for low detail, broad analysis.

Zenmap: Zenmap is the official graphical user interface for Nmap.

Denial-of-service / distributed denial-of-service tools:

Hping3: Functions as a command-line TCP/IP packet assembler; it is used to perform both Transfer Control Protocol and User Datagram Protocol floods, along with ‘Smurf’ attacks. A Smurf attack is defined as “a type of DDoS attack where an attacker floods a target victim with Internet Control Message Protocol (ICMP) echo replies." By spoofing the victim's IP address and

sending requests to a network's broadcast address, the attacker forces numerous hosts to reply to the victim simultaneously, overwhelming their network." (Cloudflare, n.d.).

1.3 Gaps identified:

- Existing tools are bulky and hard; the inner workings are hard to understand.
- AI tools cannot be used in development (security precautions)
- Existing open-source code modules are slow and often do not work when sections are used in other projects. The current workflow usually involves using a scanning tool such as Xmap, nmap or GUI-based tools like Zenmap to identify open ports and other network characteristics using tools such as Metasploit or Nessus. Then the security team member would use a tool such as LOIC, Hping3, or custom scripts as would be seen in our tool.

Research Questions:

How can a modular Python-based network port scanning tool be designed to improve the understanding of network security concepts among university students?

How can a controlled and ethical DoS stress-testing module be integrated into a network security tool to effectively demonstrate the impact of availability-based attacks in an educational setting?

Task 3: Product Concept

3.1 Product Rationale

Current network security tools such as Nmap, Masscan, and Hping3 are powerful but highly complex, making them difficult for beginners and university students to understand and use effectively. Additionally, these tools are separate utilities, meaning users must switch between multiple programs when performing network analysis and stress testing. NETWOLF aims to solve both of these problems by combining port scanning and DoS stress testing into a single, beginner-friendly, open-source Python application.

3.2 Proposed Product Features

- Network port scanner: Scans a target host for open ports and active services.
- DoS stress-testing module: Tests the availability and resilience of a network under load.
- Web-based UI: A clean and simple interface built with HTML and Tailwind CSS.
- Modular architecture: Each component can be independently extracted and reused.
- Result logging: Scan and test results are saved for documentation and review.

3.3 Programming Framework/Technology Selected

- Backend / Core Logic: Python (utilising libraries like socket and threading for efficient network interaction).
- Web Framework: Flask or FastAPI (used to bridge the Python scripts with the web interface).
- Frontend UI: HTML5 and Tailwind CSS for a lightweight, responsive web browser interface.

3.4 Innovative and Creative Aspects

The core innovation of NETWOLF lies in breaking down the "black box" nature of enterprise network tools. Instead of hiding complex processes behind bulky software, this tool merges vulnerability scanning and availability testing into one cohesive, educational platform. Because the Python codebase is deliberately modular, university students can extract the individual scanning or DoS scripts to see exactly how network packets are assembled and sent. It bridges the gap between a practical utility and an open-source teaching aid.

3.5 Potential Impact and Applications

NETWOLF has the potential to serve as an educational resource in university cybersecurity courses, allowing students to learn about port scanning and availability-based attacks in a safe and ethical environment. Beyond education, its modular design means individual components could be adapted for use in real-world penetration testing workflows, making it a valuable tool for both students and early-career security professionals.

Task 4: Production Plan

4.1 Project Phases

Phase 1 Research & Planning:

The completion of this phase involves conducting the required research and planning elements related to the project proposal. This includes reviewing and citing five or more research papers. Then moving on to defining the scope and implementation stages of the project, which involves identifying the problem (s) we aim to solve and determining our intended approach to solving the identified problem (s).

Phase 2: System Design:

This phase will be primarily focused on the creation of user interface mockups, architecture and utility planning.

This stage will be broken down into two blocks. User interface prototyping and design, which will be led by Vishesh, and system architecture and backend planning and design, which will be led by Josh.

In this stage, Vishesh will use design tools such as Figma to create high-fidelity prototypes of netwolfs user interface.

Josh will be focusing on the development of the initial utility and logic for NetWolf. This will be done by first constructing the architectural designs of the program using a variety of methods and setting up task management programs such as monday.com, and then utilising Python to create a terminal-based beta version of Netwolf.

Upon the completion of both of these blocks, we will be in a good position to move forward to phase three, which is primarily focused on the actual full system development of Netwolf.

Phase 3: Development:

In this phase, we move toward the actual primary development module for NetWolf; this involves both parties working hard to complete their required workloads effectively and on time.

For Vishesh, this will involve building the frontend based on his initial designs and mirroring all CLI functionality into a user-friendly graphical user interface.

For Josh, this stage will involve the actual creation of the backend systems behind the tool and implementing the designed architecture from phase 2. This will be broken into two distinct phases, as that is how the logic for the program is intended to be.

The first being the refinement and further development of the port scanning utility, including the different scans that can be performed, along with ensuring that data is accessible across the utility, so that the application flow is as efficient as possible, as this is one of the key improvement areas we identified when researching existing tools such as nmap and hping3.

The second phase will involve the creation and refinement of the actual denial of service utility. This stage will require in-depth analysis and testing to ensure that the tool functions as intended and as efficiently as possible. Upon the completion of these two steps, Josh will have created the functional cli based utility, and then once merged with Vishesh's gui NetWolf will be a fully functional utility.

Phase 4: Testing & Quality Assurance:

Once NetWolf is at a fully functional state, we will then delegate the testing and quality assurance portion into two more sections, with
Josh is focusing on the following items:

Item one is network scan proficiency analysis. This will be done by performing the same scans across both NetWolf and nmap and comparing results between the two to ensure that our scanner is delivering true and accurate results. The second stage will be performing an efficiency audit by again comparing Netwolf's network scanning utilities against existing tools like nmap and measuring efficiency with metrics such as scan time and resource consumption analysis.

Item two will cover the proficiency testing of the denial of service component; this will be done by again making a comparative analysis between Netwolf and existing tools such as hping3 or LOIC. This analysis will include points such as time until down, flood speed, and packet volume. Along with how the tool performs against different targets, such as a host network vs web server.

Vishesh will be responsible for the testing and debugging of the user interface and ensuring it can perform and mirror the full functionality of the CLI version.

Phase 5: Documentation & Deployment:

(Writing code comments, drafting the user manual, and final submission). This phase will be focused on documenting and deploying the finished product and ensuring proper safeguards are in place. We will equally split the creation of this documentation between the two of us. The documentation will contain two different resources:

Developer documentation, which will detail how the program works on a programmatic level and how modules can be adapted to specific ethical use cases.

And a user manual, which will contain instructions for all features and commands, both in the CLI and GUI.

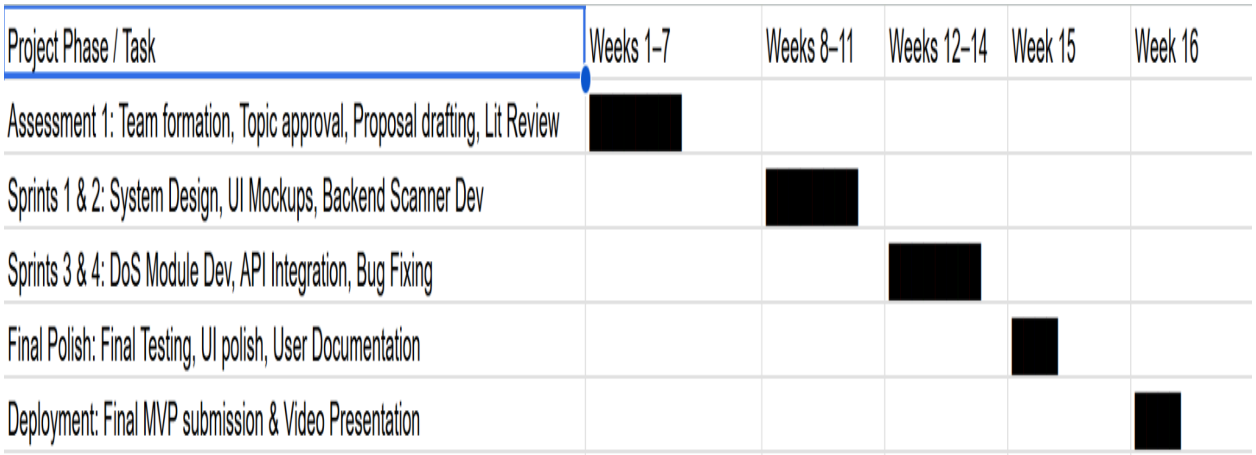
4.2 Work Breakdown Structure (WBS)

- **1.0 Research & Planning**
    - **1.1 Literature Review** - This will be completed in equal parts by both Vishesh and Josh, and will involve the reading and review of existing literature in relation to our identified area of Python-based and existing network scanning and denial of service utilities.
    - **1.2 Research Questions** - Are to be drawn from issues we identify with existing utilities.
    - **1.3 Product Concept** - This will detail the proposed product we intend to create, including the intended modules and functionality.
    - **1.4 Production Plan -** This plan will detail the steps and sprints we will perform in the production of our product.
- **2.0 Design**
    - **2.1 UI/UX Mockups (Figma/Wireframes)**
    - **2.2 System Architecture Design** - Will include data processes, storage logic, and overall application flow.
    - **2.3 Component specification -** Integrated network scanning and denial of service utilities, graphical user interface, and command-line interface.
- **3.0 Development**
    - **3.1 Backend Development: Port scanning module**: The first tool within the utility is responsible for advanced network scanning and analysis, which is then passed to 3.2.
    - **3.2 Backend Development: DoS Stress-Testing Module:** Usable as a standalone module or using network data received from 3.1, this module will be able to perform complex stress testing and denial of service operations.
    - **3.3 Frontend Development:** Development of a graphical user interface.
    - **3.4 API Integration:** Connecting the backend utility to the graphical user interface.
- **4.0 Testing**
    - 4.1 Unit testing individual modules.
    - 4.2 Integration testing - testing the performance of the GUI in relation to the CLI in terms of efficiency and accuracy.

- **5.0 Documentation & Deployment**
    - 5.1 Finalising open-source code documentation.
    - 5.2 Creating a video presentation and Minimum Viable Product report
    - 5.3 Project deployment and final review.

4.3 Gantt Chart (Timeline)

Project development chart.



Weeks 1–7:

This first phase will be broken into stages to maximize efficiency. These stages are as follows.

Team formation: This stage will cover the formation of our team and the delegation of roles.

Topic approval: Here, we will move to identifying our project scope and presenting our project topic for approval.

Literature Review: This stage involves the review of existing literature related to our project area and identifying areas that are relevant to our subject area.

Project Proposal: Here, we move on to the equal completion of our project proposal. We do this by delegating areas of responsibility and completing our fair portion of the work to ensure we meet deadlines and complete a quality proposal.

Weeks 8–11:

Sprints 1 & 2

System Design, UI Mockups: In this section, Vishesh will complete the initial design and implementation of the NetWolf graphical user interface.

Initial Backend Development of network scanning utility: Josh will develop the initial backend logic for the network scanning portion of Netwolf, after this performing initial testing before moving to the next phase.

Weeks 12–14:

Sprints 3 & 4

DoS Module Development: Here, Josh will focus on the development of the denial of service component of Netwolf.

API Integration and Bug Fixing: Here, we will both move to review the functionality and overall state of the application, along with fully configuring the connection between the backend program and the GUI.

Week 15:

Final Testing, UI polish, and User Documentation: This stage will be equally split between both Josh and Vishesh and will cover all required touch-ups and documentation finalisation before submission.

Week 16:

Final MVP submission and video presentation recording: Final presentation of the created product, including a live demonstration of its capabilities.

4.4 Team Roles

Team Member	**Role**	**Responsibilities**
Josh Arneil	Backend Developer	Josh is responsible for the development and design of the backend architecture and systems. In doing this, he will create the backbone of Netwolf, bringing it to a functional state, completing the full utility functions of the project, including the network scanning and analysis modules, along with the integrated denial of service functionalities. This will all be fully functional

		and accessible through the integrated command-line interface.
Vishesh Grover	Frontend Developer	Responsible for designing and coding the web-based UI using HTML and Tailwind CSS. Ensures a user-friendly layout, builds the result-logging dashboard, and handles user input validation on the front end.

4.5 Scrum Elements

Product Backlog: This includes all required features, bugs that have been identified and all tasks that need completion.

Sprints: These are two-week blocks, which cover each development phase.

> **Sprint one:** This includes architecture design and planning, along with frontend mockups and initial user interface development.
>
> **Sprint two:** This phase covers the actual development of the initial utility, including the development of the network scanning and analysis module along with the initial command line interface. In this phase Vishesh will work on the graphical user interface portions related to the network scanning utility.
>
> **Sprint three:** In this phase, we will begin to work on the final portions of the utility; firstly, Josh will complete the development of the integrated denial of service module and complete performance testing of the full utility so far. Vishesh will be responsible for the completion of the graphical user interface and ensuring it mirrors all functionality from the command line interface.

Daily Stand-ups: These are brief daily meetings between Vishesh and Josh where we track the progress of each sprint and the project as a whole. Along with identifying any areas of improvement or outstanding issues in the workflow.

Sprint Reviews: At the end of each sprint, we will conduct a review and evaluate our overall performance and progress for that specific sprint. We will then use these identified issues to maximise our efficiency and performance for the remaining sprints.

Sprint Retrospectives: These focus on reflecting on areas of good performance and using these identified areas and/or strategies to continue productive development through the remainder of the project.

Conclusion:

This proposal explores the development and implementation of Netwolf, an open-source utility with integrated network analysis and stress testing modules. This tool addresses key efficiency and workflow issues we have identified in existing utilities such as nmap, zenmap and hping3 and serves to solve these by containing related modules inside a single ecosystem interatible both through a command line or graphical user interface. By using a modular Python-based architecture and a web-based GUI, this utility aims to bridge the gap between existing penetration testing tools and cybersecurity education. This project proposal also includes an agile production plan to deliver the Netwolf base product that will be both highly functional and intuitive for the end user. Netwolf serves to empower students and cybersecurity professionals to ethically and safely analyse and test networks while providing a valuable asset for practical and educational applications.

References

Abu Bakar, R., & Kijsirikul, B. (2023). Enhancing network visibility and security with advanced port scanning techniques. *Sensors*, *23*(17), 7541. https://doi.org/10.3390/s23177541

Cloudflare. (n.d.). *What is a Smurf attack?* https://www.cloudflare.com/learning/ddos/smurf-ddos-attack/

Ghazali, K. W. M., & Hassan, R. (2011). Flooding distributed denial of service attacks: A review. *Journal of Computer Science*, *7*(8), 1206-1210. https://www.researchgate.net/publication/265491734_Flooding_Distributed_Denial_of_Service_Attacks-A_Review

Jafarian, J. H., Abolfathi, M., & Rahimian, M. (2023). Detecting network scanning through monitoring and manipulation of DNS traffic. *IEEE Access*, *11*, 20267-20283. https://doi.org/10.1109/ACCESS.2023.3250106

Li, X., Xie, Z., Sun, L., Qiu, Y., Xu, Z., & Liu, Z. (2026). XMap: Fast Internet-wide IPv4 and IPv6 network scanner. *arXiv*. https://arxiv.org/abs/2602.09333

Pittman, J. M. (2023). A comparative analysis of port scanning tool efficacy. *arXiv*. https://doi.org/10.48550/arXiv.2303.11282